

// PART 1 OF 4

BRONZE INGESTION

nb_bronze_ingest_sql · 19 Delta Tables · 21 Cells · 3 Lines PySpark · read_files() CTAS

```
/* Philosophy:
/* · All 19 tables ingested via SQL read_files() TVF – zero PySpark loops
/* · Python used ONCE: 3-line schema bootstrap (Cell 1) only
/* · delta.enableChangeDataFeed = true on raw_iot_events for Part 4 streaming
/* · Run Cell 1, then Cells 2-20 in any order, then Cell 21 for validation
*/
```

// nb_bronze_ingest_sql

CELL 1 – Schema Bootstrap

Python – only Python in this notebook

```
1 for schema in ['bronze', 'silver', 'gold']:
2 spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema}")
3 print("Schemas ready: bronze | silver | gold")
```

// nb_bronze_ingest_sql

CELL 2 – bronze.raw_orders

pk: order_id · expected rows: 50,000

```
1 CREATE OR REPLACE TABLE bronze.raw_orders
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_orders.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE order_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_orders' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_orders;
```

// nb_bronze_ingest_sql

CELL 3 – bronze.raw_order_lines

pk: line_id · expected rows: 128,899

```
1 CREATE OR REPLACE TABLE bronze.raw_order_lines
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_order_lines.csv',
```

```

10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE line_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_order_lines' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_order_lines;

```

```
// nb_bronze_ingest_sql
```

CELL 4 – bronze.raw_customers

pk: customer_id · expected rows: 500

```

1 CREATE OR REPLACE TABLE bronze.raw_customers
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_customers.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE customer_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_customers' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_customers;

```

```
// nb_bronze_ingest_sql
```

CELL 5 – bronze.raw_products

pk: sku · expected rows: 1,000

```

1 CREATE OR REPLACE TABLE bronze.raw_products
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_products.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE sku IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_products' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_products;

```

```
// nb_bronze_ingest_sql
```

CELL 6 – bronze.raw_inventory

pk: inventory_id · expected rows: 10,000

```
1 CREATE OR REPLACE TABLE bronze.raw_inventory
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_inventory.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE inventory_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_inventory' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_inventory;
```

```
// nb_bronze_ingest_sql
```

CELL 7 – bronze.raw_inventory_transactions

pk: txn_id · expected rows: 50,000

```
1 CREATE OR REPLACE TABLE bronze.raw_inventory_transactions
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_inventory_transactions.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE txn_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_inventory_transactions' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_inventory_transactions;
```

```
// nb_bronze_ingest_sql
```

CELL 8 – bronze.raw_shipments

pk: shipment_id · expected rows: 30,053

```
1 CREATE OR REPLACE TABLE bronze.raw_shipments
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
```

```

7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_shipments.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE shipment_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_shipments' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_shipments;

```

```
// nb_bronze_ingest_sql
```

CELL 9 – bronze.raw_carriers

pk: carrier_id · expected rows: 10

```

1 CREATE OR REPLACE TABLE bronze.raw_carriers
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_carriers.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE carrier_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_carriers' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_carriers;

```

```
// nb_bronze_ingest_sql
```

CELL 10 – bronze.raw_warehouses

pk: warehouse_id · expected rows: 10

```

1 CREATE OR REPLACE TABLE bronze.raw_warehouses
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_warehouses.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE warehouse_id IS NOT NULL;
16
17 -- row count check

```

```
18 SELECT 'bronze.raw_warehouses' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_warehouses;
```

```
// nb_bronze_ingest_sql
```

CELL 11 – bronze.raw_locations

pk: location_id · expected rows: 2,000

```
1 CREATE OR REPLACE TABLE bronze.raw_locations
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_locations.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE location_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_locations' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_locations;
```

```
// nb_bronze_ingest_sql
```

CELL 12 – bronze.raw_purchase_orders

pk: po_id · expected rows: 8,000

```
1 CREATE OR REPLACE TABLE bronze.raw_purchase_orders
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_purchase_orders.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE po_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_purchase_orders' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_purchase_orders;
```

```
// nb_bronze_ingest_sql
```

CELL 13 – bronze.raw_suppliers

pk: supplier_id · expected rows: 80

```
1 CREATE OR REPLACE TABLE bronze.raw_suppliers
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
```

```

5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_suppliers.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE supplier_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_suppliers' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_suppliers;

```

// nb_bronze_ingest_sql

CELL 14 – bronze.raw_supplier_performance

pk: perf_id · expected rows: 2,400

```

1 CREATE OR REPLACE TABLE bronze.raw_supplier_performance
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_supplier_performance.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE perf_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_supplier_performance' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_supplier_performance;

```

// nb_bronze_ingest_sql

CELL 15 – bronze.raw_returns

pk: return_id · expected rows: 5,000

```

1 CREATE OR REPLACE TABLE bronze.raw_returns
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_returns.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE return_id IS NOT NULL;

```

```
16
17 -- row count check
18 SELECT 'bronze.raw_returns' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_returns;
```

```
// nb_bronze_ingest_sql
```

CELL 16 – bronze.raw_employees

pk: employee_id · expected rows: 200

```
1 CREATE OR REPLACE TABLE bronze.raw_employees
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_employees.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE employee_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_employees' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_employees;
```

```
// nb_bronze_ingest_sql
```

CELL 17 – bronze.raw_labor_shifts

pk: shift_id · expected rows: 25,000

```
1 CREATE OR REPLACE TABLE bronze.raw_labor_shifts
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_labor_shifts.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE shift_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_labor_shifts' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_labor_shifts;
```

```
// nb_bronze_ingest_sql
```

CELL 18 – bronze.raw_dock_activity

pk: dock_id · expected rows: 15,000

```
1 CREATE OR REPLACE TABLE bronze.raw_dock_activity
2 USING DELTA
```

```

3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_dock_activity.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE dock_id IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_dock_activity' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_dock_activity;

```

```
// nb_bronze_ingest_sql
```

CELL 19 – bronze.raw_iot_events

pk: event_id · expected rows: 100,000

```

1 CREATE OR REPLACE TABLE bronze.raw_iot_events
2 USING DELTA
3 TBLPROPERTIES (
4 'delta.autoOptimize.optimizeWrite' = 'true',
5 'delta.autoOptimize.autoCompact' = 'true',
6 'delta.enableChangeDataFeed' = 'true'
7 )
8 TBLPROPERTIES (
9 'delta.autoOptimize.optimizeWrite' = 'true',
10 'delta.autoOptimize.autoCompact' = 'true',
11 'delta.enableChangeDataFeed' = 'true'
12 AS
13 SELECT
14 *,
15 current_timestamp() AS _loaded_at
16 FROM read_files(
17 'Files/raw/raw_iot_events.csv',
18 format => 'csv',
19 header => 'true',
20 inferSchema => 'true',
21 nullValue => ''
22 )
23 WHERE event_id IS NOT NULL;
24
25 -- row count check
26 SELECT 'bronze.raw_iot_events' AS table_name, COUNT(*) AS row_count
27 FROM bronze.raw_iot_events;

```

```
// nb_bronze_ingest_sql
```

CELL 20 – bronze.raw_date_dim

pk: date_key · expected rows: 1,096

```

1 CREATE OR REPLACE TABLE bronze.raw_date_dim
2 USING DELTA
3 TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
4 AS
5 SELECT

```



```
6 *,
7 current_timestamp() AS _loaded_at
8 FROM read_files(
9 'Files/raw/raw_date_dim.csv',
10 format => 'csv',
11 header => 'true',
12 inferSchema => 'true',
13 nullValue => ''
14 )
15 WHERE date_key IS NOT NULL;
16
17 -- row count check
18 SELECT 'bronze.raw_date_dim' AS table_name, COUNT(*) AS row_count
19 FROM bronze.raw_date_dim;
```

```
// nb_bronze_ingest_sql
```

CELL 21 – Full Bronze Validation

UNION ALL audit · all 19 PKs · status must be ✓ OK before Part 2

```
1 WITH audit AS (
2 SELECT 'raw_orders' AS tbl, 'order_id' AS pk,
3 COUNT(*) AS rows,
4 SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
5 FROM bronze.raw_orders
6 UNION ALL SELECT 'raw_order_lines' AS tbl, 'line_id' AS pk,
7 COUNT(*) AS rows,
8 SUM(CASE WHEN line_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
9 FROM bronze.raw_order_lines
10 UNION ALL SELECT 'raw_customers' AS tbl, 'customer_id' AS pk,
11 COUNT(*) AS rows,
12 SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
13 FROM bronze.raw_customers
14 UNION ALL SELECT 'raw_products' AS tbl, 'sku' AS pk,
15 COUNT(*) AS rows,
16 SUM(CASE WHEN sku IS NULL THEN 1 ELSE 0 END) AS pk_nulls
17 FROM bronze.raw_products
18 UNION ALL SELECT 'raw_inventory' AS tbl, 'inventory_id' AS pk,
19 COUNT(*) AS rows,
20 SUM(CASE WHEN inventory_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
21 FROM bronze.raw_inventory
22 UNION ALL SELECT 'raw_inventory_transactions' AS tbl, 'txn_id' AS pk,
23 COUNT(*) AS rows,
24 SUM(CASE WHEN txn_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
25 FROM bronze.raw_inventory_transactions
26 UNION ALL SELECT 'raw_shipments' AS tbl, 'shipment_id' AS pk,
27 COUNT(*) AS rows,
28 SUM(CASE WHEN shipment_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
29 FROM bronze.raw_shipments
30 UNION ALL SELECT 'raw_carriers' AS tbl, 'carrier_id' AS pk,
31 COUNT(*) AS rows,
32 SUM(CASE WHEN carrier_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
33 FROM bronze.raw_carriers
34 UNION ALL SELECT 'raw_warehouses' AS tbl, 'warehouse_id' AS pk,
35 COUNT(*) AS rows,
36 SUM(CASE WHEN warehouse_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
37 FROM bronze.raw_warehouses
38 UNION ALL SELECT 'raw_locations' AS tbl, 'location_id' AS pk,
39 COUNT(*) AS rows,
40 SUM(CASE WHEN location_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
41 FROM bronze.raw_locations
42 UNION ALL SELECT 'raw_purchase_orders' AS tbl, 'po_id' AS pk,
43 COUNT(*) AS rows,
44 SUM(CASE WHEN po_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
45 FROM bronze.raw_purchase_orders
46 UNION ALL SELECT 'raw_suppliers' AS tbl, 'supplier_id' AS pk,
47 COUNT(*) AS rows,
48 SUM(CASE WHEN supplier_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
49 FROM bronze.raw_suppliers
50 UNION ALL SELECT 'raw_supplier_performance' AS tbl, 'perf_id' AS pk,
51 COUNT(*) AS rows,
52 SUM(CASE WHEN perf_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
53 FROM bronze.raw_supplier_performance
54 UNION ALL SELECT 'raw_returns' AS tbl, 'return_id' AS pk,
```

```

55 COUNT(*) AS rows,
56 SUM(CASE WHEN return_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
57 FROM bronze.raw_returns
58 UNION ALL SELECT 'raw_employees' AS tbl, 'employee_id' AS pk,
59 COUNT(*) AS rows,
60 SUM(CASE WHEN employee_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
61 FROM bronze.raw_employees
62 UNION ALL SELECT 'raw_labor_shifts' AS tbl, 'shift_id' AS pk,
63 COUNT(*) AS rows,
64 SUM(CASE WHEN shift_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
65 FROM bronze.raw_labor_shifts
66 UNION ALL SELECT 'raw_dock_activity' AS tbl, 'dock_id' AS pk,
67 COUNT(*) AS rows,
68 SUM(CASE WHEN dock_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
69 FROM bronze.raw_dock_activity
70 UNION ALL SELECT 'raw_iot_events' AS tbl, 'event_id' AS pk,
71 COUNT(*) AS rows,
72 SUM(CASE WHEN event_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
73 FROM bronze.raw_iot_events
74 UNION ALL SELECT 'raw_date_dim' AS tbl, 'date_key' AS pk,
75 COUNT(*) AS rows,
76 SUM(CASE WHEN date_key IS NULL THEN 1 ELSE 0 END) AS pk_nulls
77 FROM bronze.raw_date_dim
78 )
79 SELECT
80 tbl,
81 pk AS primary_key,
82 rows,
83 pk_nulls,
84 CASE WHEN pk_nulls = 0 THEN '✓ OK' ELSE '✗ FIX' END AS status
85 FROM audit
86 ORDER BY tbl;
87
88 -- All 19 rows must show status = '✓ OK' before running Part 2.

```

```
// nb_bronze_ingest_sql
```

DEPLOYMENT NOTES

```

1 -- 1. read_files() path:
2 -- 'Files/raw/...' is relative to your Lakehouse Files section.
3 -- For ADLS Gen2: abfss://<container>@<acct>.dfs.core.windows.net/raw/<file>.csv
4
5 -- 2. inferSchema vs explicit schema:
6 -- inferSchema works for initial builds. Production → use explicit SCHEMA clause
7 -- to prevent silent type drift between pipeline runs.
8
9 -- 3. Overwrite safety:
10 -- CREATE OR REPLACE = full overwrite each run.
11 -- For incremental appends: INSERT INTO ... WHERE order_date > (SELECT MAX(...))
12
13 -- 4. raw_iot_events – Change Data Feed:
14 -- delta.enableChangeDataFeed = true is REQUIRED for Part 4 readStream.
15 -- Do NOT disable this property once the stream is running.
16
17 -- 5. Pipeline order:
18 -- [nb_bronze_ingest_sql] --success--> [nb_silver_transform_sql]

```